

PROYECTO FINAL EDA

SIMULADOR DE UN SISTEMA OPERATIVO

Profesor(a): Ariel Adara Mercado Martinez

Asignatura: Estructura de Datos y Algoritmos

Grupo: 5

Integrante(s): Gonzalez Pérez Luis Angel
Esquivel Cortés Iker Alonso
Batalla Ramírez Galileo

Equipo: Brigada 8

Semestre: SEMESTRE 2026-2

Fecha de entrega: 10/06/2026

Observaciones:

CALIFICACIÓN: _____

ESTRUCTURA DE DATOS Y ALGORITMOS

PROYECTO SIMULADOR DE SISTEMA OPERATIVO

Introducción:

Este proyecto consiste en el desarrollo de la simulación de un sistema operativo, enfocada en dos de sus funciones principales: la planificación de procesos y la administración de memoria. Para ello se implementaron distintos algoritmos de planificación, entre los que destacan FIFO, SJF (Shortest Job First) y Round Robin, así como estrategias de asignación de memoria como First Fit y Best Fit.

La implementación fue realizada en el lenguaje de programación C, utilizando diversas estructuras de datos como colas, colas circulares, listas simplemente ligadas y listas doblemente ligadas. Estas estructuras permitieron modelar de manera eficiente el comportamiento de los procesos y la memoria dentro del sistema, proporcionando una representación cercana a la utilizada en sistemas operativos reales.

Objetivo:

El objetivo principal de este proyecto es aplicar los conceptos teóricos de Estructuras de Datos en un problema práctico de gran relevancia dentro del área de sistemas operativos, en el que se analizó la forma en que diferentes algoritmos y estructuras influyen en la administración de recursos y en el desempeño del sistema.

Explicación del proyecto:

Dentro de un sistema operativo real se dan las siguientes situaciones:

- Hay muchos procesos compitiendo por el CPU
- Hay memoria limitada
- Se necesita decidir quién se ejecuta primero
- Se debe encontrar espacio disponible en RAM

En este proyecto se resuelven esos problemas, de una forma simplificada.

Primero se define un proceso, dentro del programa un proceso tiene:

- ❖ PID (identificador)
- ❖ burst_time (tiempo de CPU que necesita)
- ❖ remaining_time (cuanto le falta)
- ❖ priority (prioridad)
- ❖ memory_required
- ❖ state

Dentro de state puede ser: READY, RUNNING, BLOCKED, FINISHED

PARTE 1: PLANIFICACIÓN DE PROCESOS

En esta parte se decide quien usa el procesador

- Algoritmo FIFO

Para desarrollar el algoritmo FIFO se usó una cola (queue). Este algoritmo se encarga de asignar el orden de ejecución de los procesos, de acuerdo a su orden de llegada, siguiendo el principio FIFO (First In First Out).

Ejemplo: Suponga que se tienen los siguientes procesos

Proceso 1 llega

Proceso 2 llega

Proceso 3 llega

Entonces el orden de ejecución será:

Proceso 1

Proceso 2

Proceso 3

- SJF (Shortest Job First)

Este algoritmo hace que de una lista de procesos, se ejecute primero el de menor duración

Ejemplo: Suponga que se tienen los siguientes procesos

Proceso 1 Burst = 10

Proceso 2 Burst = 3

Proceso 3 Burst = 5

Entonces el orden de ejecución será:

Proceso 2

Proceso 3

Proceso 1

- Round Robin

Este algoritmo reparte el CPU entre todos, para ello utiliza CircularQueue, porque los procesos que no terminan en la primera ejecución, vuelven al final de la fila y repiten este proceso hasta concluir.

Ejemplo: Supongamos un Quantum = 2

Y un proceso P1 que necesita 5

P1 se ejecuta por 2 unidades y vuelve a la cola, después se vuelve a ejecutar 2 unidades y vuelve a la cola, finalmente se ejecuta 1 unidad y termina.

PARTE 2: ADMINISTRACIÓN DE MEMORIA

Donde se simula la memoria RAM, usando MemoryBlock

Ahí se define para cada bloque:

- ❖ Inicio
- ❖ Tamaño
- ❖ Si está libre
- ❖ Qué proceso lo ocupa

- First Fit

Busca el primer bloque libre suficientemente grande.

Ejemplo: Se solicitan 30

[20 libres]

[50 libres]

[80 libres]

Entonces escoge 50, porque es el primer bloque que alcanza

- Best Fit

Busca el bloque más pequeño posible que aún pueda contener el proceso

Ejemplo: Se solicitan 45

[60 libres]

[70 libres]

[50 libres]

Entonces escoge 50, porque es el bloque que desperdicia menos memoria

- Coalescencia

Se encarga de unir bloques contiguos de memoria, esto evita la fragmentación

Supongamos que existen 2 bloques de memoria libres que están juntos:

[20 libres][40 libres]

Coalescencia los junta en: [60 libres]

Ahora, todo junto trabaja de la siguiente forma:

1. Llega un proceso
PID = 1
Burst = 5
Memoria = 20
2. El memory manager busca un espacio
 - First Fit
 - Best Fit
3. El scheduler decide cuándo ejecutarlo
 - FIFO
 - SJF
 - Round Robin
4. El proceso termina
5. La memoria se libera
mm_free()
6. Los bloques libres se fusionan
mm_coalesce()

- Complejidades

stack.h (Pila) - Push / Pop / Peek: $O(1)$

queue.h (Cola) - Enqueue / Dequeue: $O(1)$

circular_queue.h (Cola Circular) - Enqueue / Dequeue: $O(1)$

linked_list.h (Lista Enlazada) - Inserción al inicio / fin: $O(1)$, Búsqueda / Eliminación: $O(n)$

doubly_linked_list.h (Lista Doblemente Enlazada) - Eliminación con referencia directa: $O(1)$, Búsqueda por valor: $O(n)$

round_robin.h (Round Robin) - Selección del próximo proceso: $O(1)$

sjf.h (Shortest Job First) - Selección con Min-Heap / Árbol de prioridad: $O(\log n)$, Selección con lista desordenada: $O(n)$

process.h / timer.h (Gestión de Procesos y Tiempo) - Actualización de estado: $O(1)$, Gestión de alarmas / hilos dormidos: $O(\log n)$ o $O(n)$

memory_manager.h (Administrador de Memoria) - Asignación (First-Fit / Best-Fit): $O(n)$, Traducción de direcciones (Paginación): $O(1)$

parser.h / algorithms.h (Análisis y Funciones Auxiliares) - Análisis de sintaxis / Tokenización: $O(m)$